Duc Nguyen - UCID: 30102065

Lab section B01

ENCM 335 - LAB 5

Exercise B:

Code:

```c
// ENCM 335 Fall 2021 Lab 4 Exercise B

// This program is defective.

#include <stdio.h>

void time_diff(int earlier_h, int earlier_min, int later_h, int later_min,
               int *diff_h, int *diff_min);
// Computes difference between two times in the same day. Assumes use of
// a 24-hour clock.

int main(void)
{
  int diff_h,diff_min;
  int later_h = 22, later_min = 35, earlier_h = 9, earlier_min = 47;
  int *p_h=&diff_h,*p_min=&diff_min;

  time_diff(earlier_h, earlier_min, later_h, later_min, p_h, p_min);

  printf("Elapsed time from %02d:%02d to %02d:%02d is ...\n",
     earlier_h, earlier_min, later_h, later_min);
  printf("... %d hour(s) and %d minute(s).\n", *p_h, *p_min);

  return 0;
}
```

```c
void time_diff(int earlier_h, int earlier_min, int later_h, int later_min,
               int *diff_h, int *diff_min)
{
  if (later_min >= earlier_min) {
    *diff_h = later_h - earlier_h;
    *diff_min = later_min - earlier_min;
  }
  else {
    *diff_h = later_h - earlier_h - 1;
    *diff_min = later_min + 60 - earlier_min;
  }
}
```

Output:

```
monsi@LAPTOP-M9ONH6KV ~/ENCM_335/lab5
$ ./a.exe
Elapsed time from 09:47 to 22:35 is ...
... 12 hour(s) and 48 minute(s).
```

Exercise C:

Code:

```c
// ENCM 335 Fall 2021 Lab 5 Exercise C

#include <stdio.h>

void display_array(const char *label, const double *x, size_t n);
// REQUIRES
//    label points to the beginning of a string.
//    Elements x[0], ... x[n-1] exist.
// PROMISES
//    label is printed, followed by values of  x[0], ... x[n-1], all on
//    one line, using %4.2f format for the doubles. If n == 0, the line
//    of output points out that fact.

void reverse(double *x, size_t n);
// REQUIRES
//     n > 0.
//     Array elements x[0] ... x[n - 1] exist.
// PROMISES
//     Order of elements x[0] ... x[n - 1] has been reversed.
//     (So the new x[0] value is the old x[n - 1] value, and so on.)

int increasing(const double *x, size_t n);
// REQUIRES
//     n > 0.
//     Array elements x[0] ... x[n - 1] exist.
// PROMISES
//     Return value is 1 if n == 1.
//     If n > 1, return value is 1 if all of a[0] < a[1] ... a[n-2] < a[n-1]
//     are true, otherwise return value is 0.

// In Step 3, add a function prototype and function interface comment
// for max_element here.

double max_element(const double *x, size_t n);
// REQUIRES
//     n > 0.
//     Array elements x[0] ... x[n - 1] exist.
// PROMISES
//     Return the maximum element in the array of doubles

int main(void)
{
  double test1[ ] = {1.1, 2.2, 3.3, 4.4, 5.5};
  double test2[ ] = {-0.5, -1.0, -1.5, -2.0, -2.5, -3.0};

  printf("Some quick checks of display_array ...\n");
  display_array("  all of test1:", test1, 5);
  display_array("  none of test1:", test1, 0);
  display_array("  last 3 elements of test1:", &test1[2], 3);
  display_array("  all of test2:", test2, 6);

  printf("\nTwo tests of reverse ...\n");
  reverse(test1, 5);
  display_array("  test1 after reversing:", test1, 5);
  reverse(test2, 6);
  display_array("  test2 after reversing:", test2, 6);

  double test3[ ] = {1.5, 1.25, 2.5, 3.5, 4.5, 5.5, 5.25};
  double test4[ ] = {1.0, 2.0, 3.0, 3.0, 4.0, 5.0};
```

```
    int inc;
    printf("\nSome tests of increasing ...\n");
    display_array("  for these numbers:", test3, 6);
    inc = increasing(test3, 6);
    printf("  expected return value is 0, actual value is %d\n\n", inc);
    display_array("  for these numbers:", &test3[1], 5);
    inc = increasing(&test3[1], 5);
    printf("  expected return value is 1, actual value is %d\n\n", inc);
    display_array("  for these numbers:", &test3[1], 6);
    inc = increasing(&test3[1], 6);
    printf("  expected return value is 0, actual value is %d\n\n", inc);
    display_array("  for these numbers:", test4, 6);
    inc = increasing(test4, 6);
    printf("  expected return value is 0, actual value is %d\n\n", inc);

    // In Step 3, add some tests for max_element here.
    double test5[]={-3.2,-1.5,-4.7,-5.5,-2.2,-0.9};
    double test6[]={3.2,1.5,5.5,2.2,0.9};
    double test7[]={3.2,1.5,-4.7,-5.5,2.2,0.9};
    double test8[]={-3.2,1.5,-5.5,0.9,2.2};
    double max;

    printf("\nSome tests of max_element ...\n");
    display_array("  for these numbers:", test5, 6);
    max=max_element(test5, 6);
    printf("  expected return value is -0.90, actual value is %.2f\n\n", max);
    display_array("  for these numbers:", test6, 5);
    max=max_element(test6, 5);
    printf("  expected return value is 5.50, actual value is %.2f\n\n", max);
    display_array("  for these numbers:", test7, 6);
    max=max_element(test7, 6);
    printf("  expected return value is 3.20, actual value is %.2f\n\n", max);
    display_array("  for these numbers:", test8, 5);
    max=max_element(test8, 5);
    printf("  expected return value is 2.20, actual value is %.2f\n\n", max);

    return 0;
}

void display_array(const char *label, const double *x, size_t n)
{
    size_t i ;
    printf("%s", label);
    if (n == 0)
      printf("  [no contents to print]\n");
    else {
      for(i = 0; i < n ; i++)
        printf(" %4.2f", x[i] );
      printf("\n");
    }
}

void reverse(double *x, size_t n)
{
    // Replace this comment with your code.
    int i;
    double a;
    for (i=0;i<n/2;i++){
        a=x[i];
        x[i]=x[n-1-i];
        x[n-1-i]=a;
    }

}

int increasing(const double *x, size_t n)
{
    // Replace this comment with your code.
    int i,a;
    if (n==1)
        return 1;
    else
        for (i=0;i<n-1;i++)
            if (x[i]<x[i+1])
                a=1;
            else{
                a=0;
                break;
            }
    return a;
}
```

```
143  // In Coding Step 3, add a function definition for max_element here.
144  double max_element(const double *x, size_t n){
145      int i;
146      double max=x[0];
147      for (i=0;i<n;i++){
148          if (max<x[i])
149              max=x[i];
150      }
151      return max;
152  }
153
```

Output:

```
monsi@LAPTOP-M9ONH6KV ~/ENCM_335/lab5
$ ./a.exe
Some quick checks of display_array ...
  all of test1: 1.10 2.20 3.30 4.40 5.50
  none of test1:  [no contents to print]
  last 3 elements of test1: 3.30 4.40 5.50
  all of test2: -0.50 -1.00 -1.50 -2.00 -2.50 -3.00

Two tests of reverse ...
  test1 after reversing: 5.50 4.40 3.30 2.20 1.10
  test2 after reversing: -3.00 -2.50 -2.00 -1.50 -1.00 -0.50

Some tests of increasing ...
  for these numbers: 1.50 1.25 2.50 3.50 4.50 5.50
  expected return value is 0, actual value is 0

  for these numbers: 1.25 2.50 3.50 4.50 5.50
  expected return value is 1, actual value is 1

  for these numbers: 1.25 2.50 3.50 4.50 5.50 5.25
  expected return value is 0, actual value is 0

  for these numbers: 1.00 2.00 3.00 3.00 4.00 5.00
  expected return value is 0, actual value is 0


Some tests of max_element ...
  for these numbers: -3.20 -1.50 -4.70 -5.50 -2.20 -0.90
  expected return value is -0.90, actual value is -0.90

  for these numbers: 3.20 1.50 5.50 2.20 0.90
  expected return value is 5.50, actual value is 5.50

  for these numbers: 3.20 1.50 -4.70 -5.50 2.20 0.90
  expected return value is 3.20, actual value is 3.20

  for these numbers: -3.20 1.50 -5.50 0.90 2.20
  expected return value is 2.20, actual value is 2.20
```